

Evaluating LLM-Generated Ontologies in a Low-Resource Clinical Domain: The BrainIT Case

Mohamed Abdellah | Dr. David Corsar

PROBLEM & MOTIVATION

An ontology is a structured, machine-readable way of organising knowledge. It acts as a digital map of a domain, defining key concepts and the logical relationships between them. While essential for clinical data integration and reasoning, ontology construction remains **slow, expert-dependent, and difficult to scale**.

In the Neuro-ICU domain, few public ontologies exist. The **BrainIT consortium** curates multi-site traumatic brain injury monitoring data across 22 European centres, but the data model is closed and defined only as a database schema, meaning sharing and extending it with new data is a largely labour-intensive effort.

LLMs are increasingly being used in such knowledge engineering tasks, therefore this project investigates the following research question:

Can large language models help reduce the knowledge engineering bottleneck by automatically extracting ontological structure from clinical literature?

AIM & OBJECTIVES

Aim: To investigate the extent to which LLMs can reconstruct a domain ontology from clinical literature, and to identify which prompting strategies, post-processing techniques, and evaluation conditions most improve performance.

- Objective 1:** Design and implement an **end-to-end knowledge engineering pipeline** that ingests clinical literature and produces structured ontologies using LLM-driven extraction
- Objective 2:** Evaluate LLM capability to extract **classes, relations, and class hierarchies**
- Objective 3:** Compare **Zero-Shot, One-Shot (MMR), and phased Few-Shot** prompting strategies
- Objective 4:** Measure each processing stage's contribution through **per-stage ablation analysis**

WEB APPLICATION

The framework includes a purpose-built Flask web application that serves as both an experiment workbench and a results analysis platform. Users can configure and launch extraction runs, explore generated ontologies through an interactive graph, and compare results across different configurations.

CORE FEATURES

3 Prompting Strategies

Zero-Shot, One-Shot (MMR), Few-Shot

3 Pipeline Modes

Strict, Guided, Schema-Completed

7 LLM Providers

GPT-4o, Claude, Gemini, Llama & more

17 Pipeline Toggles

Fine-grained ablation control

Interactive Graph

Explore ontologies visually

Batch Comparison

Pause, resume & skip controls

Dual-LLM Design

Separate extraction & reasoning models

Per-Stage Ablation

Metrics snapshot at each pipeline step

Run configuration

Prompting method

Zero-ShotOne-ShotFew-Shot

Choose prompting strategy

Pipeline mode

StrictGuidedSchema-Completed

Select a pipeline mode

Paper

Paper 1 — Piper 2003 — The BrainIT Group: 1

Pick a BrainIT paper

LLM provider

GPT-4o-miniGPT-4oClaude Haiku 4.5Gemini 2.0 FlashLlama 3.1 8BMistral 7BDeepSeek V3

Select LLM provider

Evaluation settings

Gold-vocabulary-only

Configure evaluation settings

Advanced / Experimental

Reasoning LLM (for Models 3 & 4)

OpenAI (GPT-4o-mini)OpenAI (GPT-4o)DeepSeek ReasonerAdvanced: reasoning LLM

Add run

Reset

Building your ontology

Run ID: 28268325-233965-98273c4c

Progress

Running

Cancel

Processing...

Chunk 1 of 3

33%

Knowledge graph - Classes & object properties

Producing triples

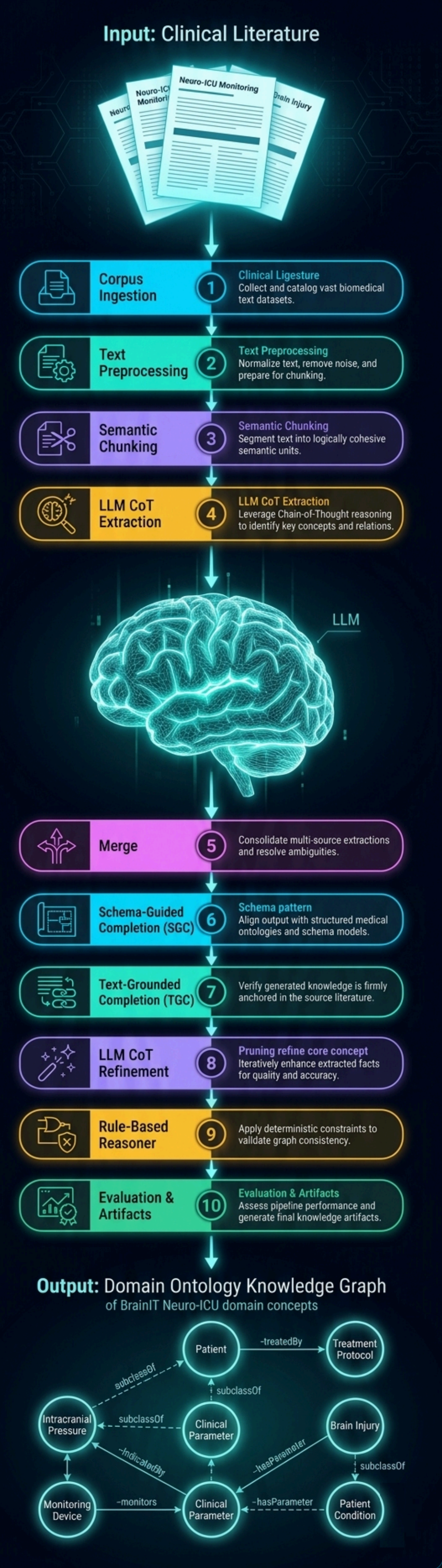
PatientreceivesTherapyTherapy

Ontology Graph

70 classes58 relations27 hierarchy

Outcome Score (GOS)Clinical OutcomeSecondary InjuryEUSID GradeAcute Hypertensive EpisodesGOS AssessmentTraumatic Brain InjuryPupil AssessmentConditionIntracranial PressureOutcomeBarbituratesFirst-line TreatmentMechanical VentilationPatientPressure Reactivity IndexPatientDemographicsHypotensionSevere Traumatic Brain Injury (STBI)Intensive Care

PIPELINE ARCHITECTURE



METHODS — PIPELINE STAGES

The pipeline transforms clinical literature into a structured ontology through a sequence of ten automated stages. Each stage builds on the previous output, progressively refining raw text into validated, domain-specific knowledge representations.

- Corpus Ingestion** — load clinical papers, strip inline citations and reference sections, expand domain abbreviations using a 35-entry neuro-ICU dictionary
- Text Preprocessing** — normalise text encoding and whitespace, apply scope filtering to remove non-domain content such as author bios and funding disclosures
- Semantic Chunking** — split documents into overlapping 2000-token chunks with section-type classification (high / medium / low priority); reference sections are skipped entirely
- LLM CoT Extraction** — per-chunk Chain-of-Thought prompting with mandatory evidence grounding; **MMR retriever** selects diverse few-shot examples; **Medical NER** via ScispaCy anchors domain vocabulary
- Merge** — combine per-chunk extractions into a unified ontology; deduplicate classes via canonical alias mapping, aggregate provenance and evidence across sources
- Schema-Guided Completion (SGC)** — fills structural gaps by comparing the extracted ontology against expected schema patterns and adding missing relations and hierarchy edges
- Text-Grounded Completion (TGC)** — multi-pass enrichment from source text using Chain-of-Layer taxonomy building to discover classes and hierarchy missed during initial extraction
- LLM CoT Refinement** — a separate reasoning LLM reviews the full ontology, prunes hallucinated concepts lacking textual evidence, and repairs structural inconsistencies
- Rule-Based Reasoner** — 11-step deterministic cleanup applying domain axioms, out-of-scope pruning, relation vocabulary standardisation, hierarchy cycle repair, and orphan rescue
- Evaluation & Artifacts** — computes coverage, precision, and recall against the gold standard; generates per-stage ablation tables, error taxonomy reports, and full run metadata for reproducibility

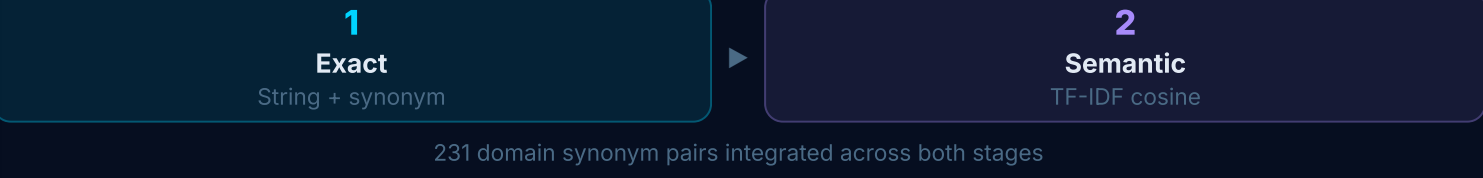
Every extracted concept must be backed by a **direct textual quote** from the source — this evidence requirement is the primary quality gate throughout the pipeline. The system employs a **dual-LLM design**: one model handles extraction while a separate reasoning model reviews and refines the output. Three progressive pipeline modes — **Strict** (raw extraction only), **Guided** (+ vocabulary guardrails and Medical NER), and **Schema-Completed** (+ ontology completion and rule-based reasoning) — give researchers fine-grained control over how much automated post-processing is applied to the LLM output.

EXPECTED OUTCOMES & EVALUATION

Few-Shot prompting is expected to outperform Zero-Shot and One-Shot because worked examples help the LLM anchor on the correct ontological granularity. Class extraction is anticipated to be strongest, as classes map to noun phrases LLMs readily identify. Relation extraction should be moderate, requiring recognition of implicit semantic connections. Hierarchy reconstruction is likely weakest, demanding deeper structural reasoning.



Extracted ontologies are evaluated against the **BrainIT gold standard** via **2-stage matching** with synonym awareness:



Metrics: Coverage, precision, and recall are computed at each pipeline stage to assess both output quality and the marginal contribution of each processing component.

LIMITATIONS

- Cost constraints limited full cross-LLM evaluation across all strategy, mode, and provider combinations
- LLM outputs are non-deterministic, introducing variability between runs even with identical configurations
- The corpus is limited to 11 BrainIT publications, which may not capture the full breadth of domain knowledge
- Hardware constraints prevented testing of locally hosted biomedical LLMs that may better capture clinical terminology

FUTURE WORK

- Expand to larger LLMs to test whether post-processing remains necessary
- Develop a domain-specific NER model for Neuro-ICU vocabulary
- Transfer the framework to other clinical domains
- Add human-in-the-loop editing so domain experts can iteratively refine LLM-generated ontologies
- Fine-tune smaller LLMs on ontology extraction tasks to reduce API dependency and cost

TECH STACK

- Python 3.11
- Flask
- LLM APIs
- RDFLib
- ScispaCy
- Cytoscape.js
- scikit-learn
- JavaScript
- pytest

ACKNOWLEDGEMENTS

My sincere thanks to **Dr. David Corsar** for his guidance and support throughout this project, and to **Dr. Laura Moss** from BrainIT for her valuable domain expertise and insight.